STUDENT PERFORMANCE ANALYTICS SYSTEM

Project Report | MySQL Database Design and Python Data Analysis

PART A - DATABASE DESIGN • PART B - CSV ANALYSIS

1. Introduction

A college needs to store student, department, subject and marks data consistently, then use the records to understand academic performance. This project defines a relational database in MySQL and uses Python with Pandas and plotting libraries to clean, combine, summarize and visualize the same information.

Objectives	Dataset scope
Maintain linked records; apply keys and validation rules; calculate student and subject performance.	4 departments, 30 students, 6 subjects and 180 mark records from four CSV datasets.

2. Database Design

The schema has four tables. The marks table acts as the link between students and subjects; department details are stored once and referenced by students.

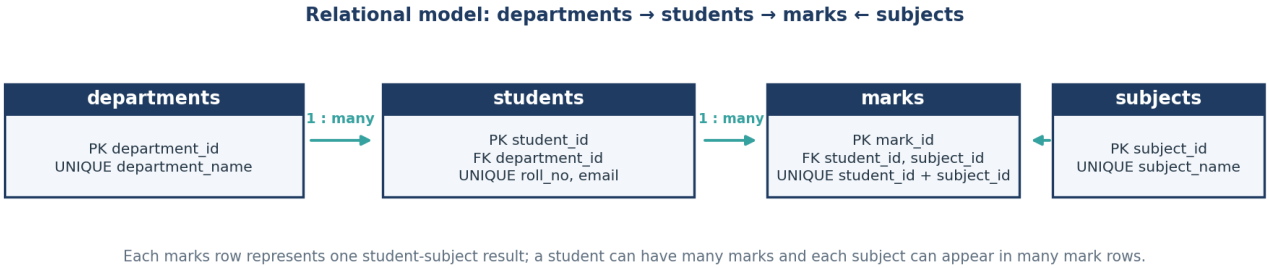


Figure 1. Entity relationships inferred from the submitted CREATE TABLE statements.

Table	Keys and relationships	Validation / purpose
departments	PK department_id	department_name is NOT NULL and UNIQUE
students	PK student_id; FK department_id	roll_no and email UNIQUE; semester 1-8; default semester 1
subjects	PK subject_id	subject_name UNIQUE; semester 1-8; credits 1-6; default 3
marks	PK mark_id; FK student_id and subject_id	marks 0-100; UNIQUE(student_id, subject_id)

Constraints and normalization

PRIMARY KEY identifies each row; FOREIGN KEY protects references; UNIQUE prevents duplicate roll numbers, emails, names and student-subject pairs; NOT NULL requires essential values; CHECK restricts semester, credits and marks ranges; DEFAULT supplies semester or credit values when omitted.

The layout follows a practical 3NF-oriented separation: department attributes, student attributes, subject attributes and marks are stored in their own tables. Each row stores atomic values, and non-key details are kept with the entity they describe, reducing repeated data and update anomalies.

Normalization steps: 1NF - atomic fields and no repeating subject columns; 2NF - marks are stored as individual student-subject rows, with marks dependent on that record; 3NF - department and subject descriptions are stored in lookup tables and referenced by IDs.

3. SQL Work

The SQL script targets MySQL 8.0.16+ and creates the database, four tables, sample data, analytical queries and three stored functions.

Query / function	Purpose and SQL technique
Joined marks report	JOINs marks to students, departments and subjects to produce readable student-subject results.
Subject statistics	GROUP BY with AVG, MAX and MIN gives mean, highest and lowest marks for each subject.
Department toppers	CTEs calculate each student average; DENSE_RANK returns the leading student(s), including ties.
At-risk students / department averages	Marks below 40 identify students failing at least one subject; LEFT JOIN retains departments even if no marks exist.
Stored functions	get_grade classifies a mark; get_percentage returns a student average; get_result returns PASS, FAIL or N/A.

Integrity tests. The SQL file includes commented examples for PRIMARY KEY, UNIQUE, NOT NULL, FOREIGN KEY, CHECK, duplicate student-subject and DEFAULT behavior. They are designed to be run individually; the submitted files do not contain executed constraint-error screenshots.

4. Python Work

CSV file	Rows × columns	Role
departments.csv	4 × 2	Department lookup
students.csv	30 × 6	Student identity and department
subjects.csv	6 × 4	Subject, semester and credits
marks.csv	180 × 4	Marks by student and subject

Processing pipeline. Pandas reads the CSVs, coerces marks to numeric, removes duplicate student-subject pairs, fills missing marks with the subject mean, and merges the four datasets. A pivot table is then used to calculate totals, percentages, grades, pass/fail status and ranks. The notebook exports student_report.csv and builds charts with Matplotlib and Seaborn.

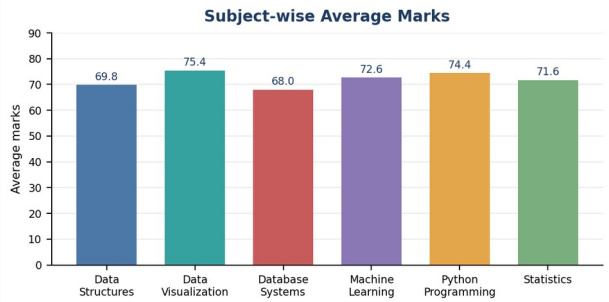
Output check	Finding from the submitted data
Top student	Yash Gupta (Information Technology): 83.17%; Sakshi Jha and Ira Singh tie for rank 2 at 82.50%.
Overall result	24 of 30 students Pass (80%); 6 students Fail (20%) under the notebook rule that every subject mark must be at least 40.
Department averages	Electronics is highest at 75.55%; Data Science is lowest at 70.33%.
Subject averages	Data Visualization is highest at 75.37%; Database Systems is lowest at 68.00%.

Implementation gaps to address: The assignment asks for NumPy and a menu-driven Python program, but the notebook does not explicitly import NumPy or implement a menu loop. Also, SQL and Python use different grade cutoffs: SQL starts A+ at 90 and A at 80, while Python starts A+ at 85 and A at 75. The SQL script ends with DROP DATABASE student_analytics; running it top-to-bottom removes the database after creating and querying it. Remove/comment out that final statement for normal use.

5. Results and Charts

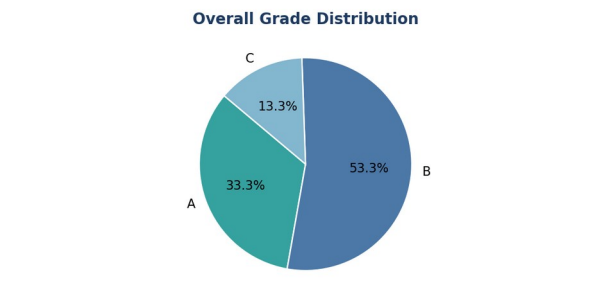
The four visuals below are based on the notebook's analysis steps and the marks supplied in the SQL inserts. The current Python grade bands are A+ ≥ 85, A ≥ 75, B ≥ 65, C ≥ 50, D ≥ 40 and F below 40.

Chart 1 | Subject averages



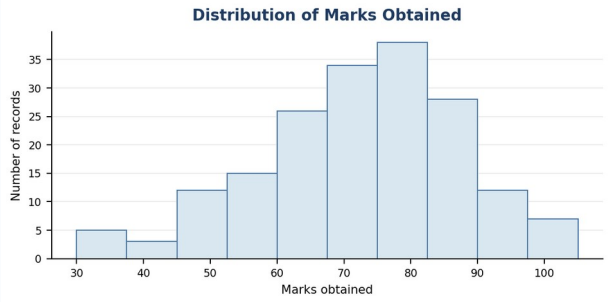
Data Visualization has the highest mean (75.37), followed by Python Programming (74.43). Database Systems is lowest at 68.00.

Chart 2 | Grade mix



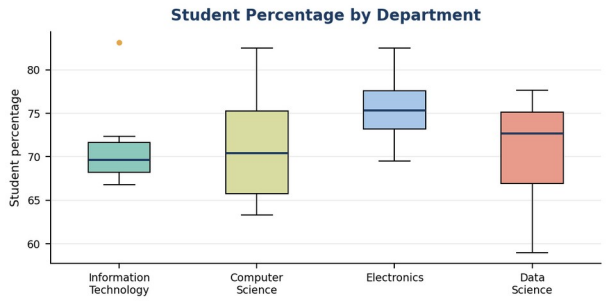
B is the largest group (16 students, 53.3%), followed by A (10, 33.3%) and C (4, 13.3%).

Chart 3 | Raw marks distribution



Scores cluster mostly around the mid-60s to mid-80s. The observed range is 30-100, so a small low-score tail deserves attention.

Chart 4 | Department percentages



Electronics has the highest average percentage (75.55%); Data Science is lowest (70.33%). Box widths show variation among students within each department.

6. Conclusion and Future Improvements

The project demonstrates an end-to-end academic analytics workflow: relational tables maintain structured records, constraints support data integrity, SQL queries summarize results, and Pandas transforms the marks into student-level reports and visualizations. The results identify a leading department and subject, show the grade mix, and surface students needing academic support.

Priority	Recommended improvement
Correctness	Use one shared grade-threshold policy in SQL and Python; add tests for boundary values such as 39.99, 40, 74.99 and 75.
Safe execution	Remove the final DROP DATABASE statement from the normal run script; keep destructive cleanup in a separate reset script.
Assignment completeness	Add a menu loop (for example: show topper, subject summary, department analysis, export report), use NumPy explicitly where appropriate, and capture real constraint-test screenshots.

Overall conclusion: The core database and analysis workflow is present in the submitted files. Aligning the grade rules and preserving the database after script execution are the most important fixes before final submission.